15,471,876 members

Sign in

articles Q&A forums stuff lounge ?

Search for articles, questions,

Auto Flow Buttons Panel



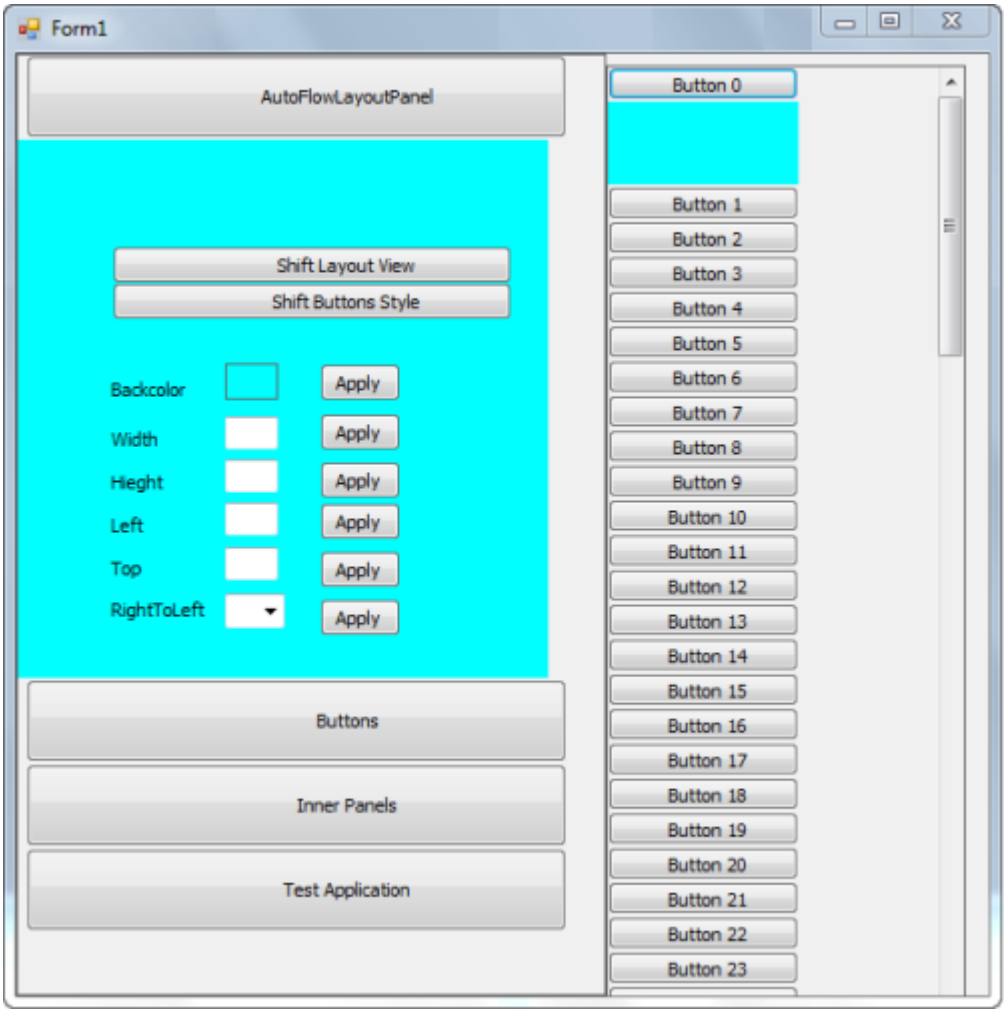
Yasser Sobhy

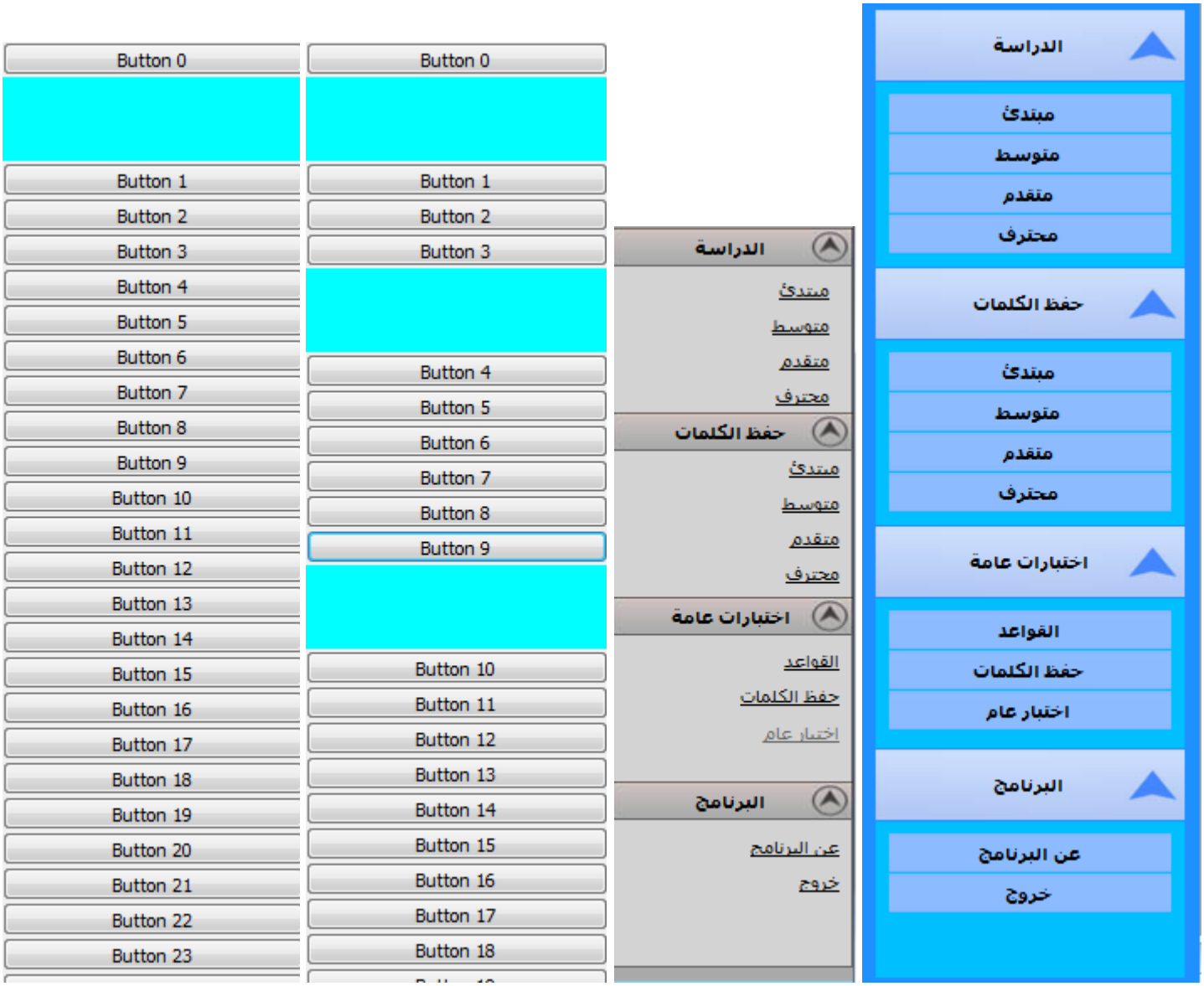
26 Sep 2011 CPOL

Rate me: 4.59/5 (11 votes)

A .NET control to control the layout of categorized sections in your application.

Download demo and source code - 119 KB


Form1
AutoFlowLayoutPanel
Shift Layout View
Shift Buttons Style
Backcolor
Apply
Width
Apply
Hieght
Apply
Left
Apply
Top
Apply
RightToLeft
Apply
Buttons
Inner Panels
Test Application
Button 0
Button 1
Button 2
Button 3
Button 4
Button 5
Button 6
Button 7
Button 8
Button 9
Button 10
Button 11
Button 12
Button 13
Button 14
Button 15
Button 16
Button 17
Button 18
Button 19
Button 20
Button 21
Button 22
Button 23



Don't forget to vote. Your vote helps to keep adding features.

Introduction

In this article, I will show you how to create a .NET control that can be used to categorize sections in your application. This control can also be used in many other ways. Just look at the source code and the demo to get a general view of how it works, then use it the way you like.

If you have any problems with this control or you want to add a feature, report a bug, or you want to submit your own changes, you are welcome and I will be happy to help you.

Background

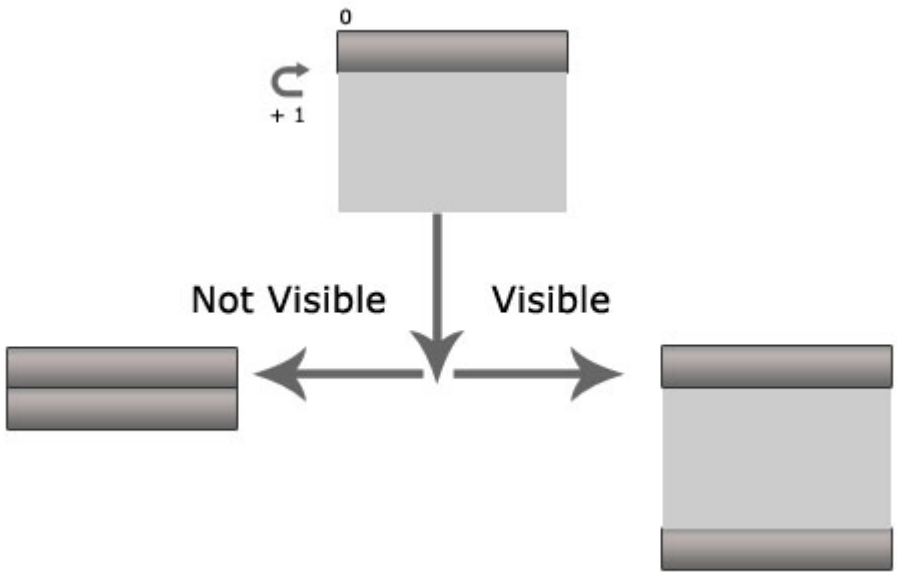
The idea to create this control came to me an year ago when I was developing an application to teach English for Arabic users, and I needed to categorize the sections of my application (study, vocabulary,

tests, and so on). I searched the internet for such a control, but I didn't find any control, code, or article that met my needs. So I decided to implement a control myself and put its code here to be available for everyone.

How it works

The corner stone of this control is to know how buttons and their panels are arranged; this is done in certain steps and these steps are:

1. we know that every button has a panel associated with it
2. we put all the buttons of our control into a `list` class
3. we set the first button's top to 0
4. we set the first button's panel's top to *the bottom of the first button + 1*
5. every new button should be placed at the bottom of the panel of the button that precedes it
6. we do a loop to check and see if the panel associated to every button is visible or not, and if it is visible, we set the next button's top to the bottom of that panel, and if the panel is not visible, we set the next button's top to the bottom of the button itself.



MultiView, SingleView styles and ArrangeAll method

In the `MultiView` style, the user can explore and see the contents of **one or more panels**; in the `SingleView` style, the user can explore and see the contents of **only one panel**.

The `ArrangeAll` method arranges all the buttons in our list according to the steps mentioned above, but with an exception, and this exception is: in the `MultiView` style, the user can explore more than one panel at a time, and in the `SingleView` style, the user can explore only one panel at a time, so in `SingleView`, we must create a **variable** to store the previously viewed button and use that variable to hide the previously viewed panel before we show the new panel contents, so when the user clicks a

button, the `ArrangeAll` method is called on the `OnClick` event of that button and we send that button to the `ArrangeAll` method. The method does the following: if the style is `MultiView`, in the `Visible` state, if the the sender button's panel is `false`, then:

- set the `Visible` state of the the sender button's panel to `true`
- and if not, set the `Visible` state of the the sender button's panel to `false`
- apply the sixth step mentioned above
    - if the style is `SingleView`
    - if the `Visible` state of the sender button's panel is `false` then:
- set the `Visible` state of the sender button's panel to `true`
- and if not, set the `Visible` state of the sender button's panel to `false`
- hide the previously viewed button's panel (because we need only one panel to be viewed a time)
- apply the sixth step mentioned above

Consider this code:

VB Shrink ▲

```
Friend Sub ArrangeAll(ByVal sender As YSSPButton, ByVal e As System.EventArgs)
    If PanelVariables.LayoutVeiw = LayoutView.MultiView Then
        Dim C As YSSPButton
        sender.InnerPanel.Visible = Not sender.InnerPanel.Visible
        sender.InnerPanel.Top = sender.Bottom + 1
        For I As Int16 = 1 To Buttons.Count - 1
            C = InnerFlowButtons(I - 1)
            If Not C Is Nothing Then
                If C.InnerPanel.Visible = True Then
                    InnerFlowButtons(I).Top = C.InnerPanel.Bottom + 1
                    ' put the button after the previous button's panel
                Else
                    InnerFlowButtons(I).Top = C.Bottom + 1
                    ' put the button after the previous button
                End If
            End If
            InnerFlowButtons(I).InnerPanel.Top = InnerFlowButtons(I).Bottom + 1
            ' every panel must be placed directly after its button
        Next
    Else
        If sender.InnerPanel.Visible = True Then Return
        ' in single view if current visible button is clicked
        ' there is no need to arrange all buttons

        Dim C As YSSPButton
        ' to let the user see panel contents
        sender.InnerPanel.Visible = True
        ' every panel must be placed directly after its button
        sender.InnerPanel.Top = sender.Bottom + 1
        ' hide the panel of the previous viewed button
        PreViewedButton.InnerPanel.Visible = False
        ' every panel must be placed directly after its button
        PreViewedButton.InnerPanel.Top = sender.Bottom + 1
        If PanelVariables.LayoutVeiw = _
```

```
                WindowsApplication2.YSSPAutoFlowButtonsPanel.LayoutView.SingleView Then
            If PanelVariables.ButtonsStyle = _
                  WindowsApplication2.YSSPAutoFlowButtonsPanel.ButtonsStyle.Graphical Then
                PreViewedButton.BackgroundImage = StateImages.MainImage
                ' set the image of the previous viewed button
                ' to the main state image not the seleced state image
            End If
        End If
        PreViewedButton = sender ' to be used in the next call

        For I As Int16 = 1 To Buttons.Count - 1
            C = InnerFlowButtons(I - 1)
            If Not C Is Nothing Then
                If C.InnerPanel.Visible = True Then
                    InnerFlowButtons(I).Top = C.InnerPanel.Bottom + 1
                    ' put the button after the previous button's panel
                Else
                    InnerFlowButtons(I).Top = C.Bottom + 1
                    ' put the button after the previous button
                End If
            End If
            InnerFlowButtons(I).InnerPanel.Top = InnerFlowButtons(I).Bottom + 1
            ' every panel must be placed directly after its button
        Next
    End If
End Sub
```

Using the Code

Our class here contains some other classes to control its style and behavior, and these classes are:

- `YSSPAutoFlowButtonsPanel`: this is the class of the panel itself.
- `PanelVariables`: this class contains general information that can be applied to all inner panels and buttons, and these settings are used to **add** new buttons and inner panels with the same style of the other buttons and panels currently inside the main panel.
- `Buttons`: this class is used to apply the following settings to an individual button; if you want to apply a setting to all buttons, you can do it using the `ButtonsSettings` class.

VB

```
Public Sub Add(ByVal Button As YSSPButton)

' these methods are not tested or debuged yet
Public Sub Remove(ByVal index As Integer)

Public Sub Remove(ByVal Button As YSSPButton)

Public Sub Hide(ByVal Index As Integer)

Public ReadOnly Property Count() As Integer

Public Sub SetWidth(ByVal Index As Integer, ByVal Width As Integer)
```

```
Public Sub SetHeight(ByVal Index As Integer, ByVal Height As Integer)

Public Sub Insert(ByVal Index As Integer, ByVal Button As YSSPButton)

Public Sub InsertRange(ByVal Start As Integer, ByVal Buttons As List(Of YSSPButton))

Public Sub Clear()
```

This class is not fully implemented or tested and needs more work; if you like to implement this class, you are welcome.

- `Panels`: this class is used to apply the following settings to an individual panel; if you want to apply a setting to all panels, you can do it using the `PanelsSettings` class.

VB

```
Public Sub SetPanelWidth(ByVal Index As Integer, ByVal Width As Integer)
Public Sub SetPanelHeight(ByVal Index As Integer, ByVal Height As Integer)
```

This class is not fully implemented or tested and needs more work; if you like to implement this, class you are welcome.

- `ButtonsSettings`: this class is used to apply the following settings to all buttons; if you want to apply a setting to an individual button, you can do so using the `Buttons` class.

VB

```
Font
Cursor
ForeColor
BackColor
Width
Height
Left
RightToLeft
ImageAlign
ImageIndex
ImageKey
ImageList
TextAlign
TextImageRelation
Parent
```

- `PanelsSettings`: this class is used to apply the following settings to all panels; if you want to apply a setting to an individual panel, you can do so using the `Panels` class.

VB

```
BackgroundImage
BackgroundImageLayout
Cursor
ForeColor
BackColor
Height
```

```
width
Left
RightToLeft
Parent
```

- `Settings`: this class is used to get or set the general settings for the main panel.
- `StateImages`: this class is used to provide images for different states for inner buttons.
- `LayoutView`: this class is used to set or get the layout view of the panels.

VB

```
SingleView ' only one category can be viewed
MultiView ' many categories can be viewed at the same time
```

- `ButtonsStyle`: this class is used to get or set the style of the inner buttons.

VB

```
Graphical ' graphical style
Colors ' not implemented yet
System ' system style
```

- `YSSPButton`: this class is the button used in our control; it inherits the `Button` class and overrides the following subs:

VB Shrink ▲

```
Protected Overrides Sub OnParentChanged(ByVal e As System.EventArgs)
    MyBase.OnParentChanged(e)
    InnerPanel.Parent = Me.Parent
    ' change the innerpanel's parent to the button's parent
End Sub
Protected Overrides Sub OnLocationChanged(ByVal e As System.EventArgs)
    MyBase.OnLocationChanged(e)
    InnerPanel.Top = Me.Bottom + 1
    ' every panel must be placed directly after its button
End Sub
Protected Overrides Sub OnSizeChanged(ByVal e As System.EventArgs)
    MyBase.OnSizeChanged(e)
    InnerPanel.Top = Me.Bottom + 1
    ' every panel must be placed directly after its button
End Sub

Protected Overrides Sub OnMouseEnter(ByVal e As System.EventArgs)
    'return if this button is the selected one (Don't change its state)
    If PanelVariables.ButtonsStyle = ButtonsStyle.Graphical Then
        If PanelVariables.LayoutVeiw = LayoutView.SingleView Then
            If PanelVariables.ButtonsStyle = ButtonsStyle.Graphical Then
                If Me.InnerPanel.Visible = True Then
                    Exit Sub
                End If
            End If
        End If
        Me.BackgroundImage = StateImages.OverImage
    End If
    MyBase.OnMouseEnter(e)
```

```
End Sub

Protected Overrides Sub OnMouseLeave(ByVal e As System.EventArgs)
    MyBase.OnMouseLeave(e)
    If PanelVariables.ButtonsStyle = ButtonsStyle.Graphical Then
        'return if this button is the selected one (Don't change its state)
        If PanelVariables.ButtonsStyle = ButtonsStyle.Graphical Then
            If PanelVariables.LayoutVeiw = LayoutView.SingleView Then
                If PanelVariables.ButtonsStyle = ButtonsStyle.Graphical Then
                    If Me.InnerPanel.Visible = True Then
                        Exit Sub
                    End If
                End If
            End If
        End If
        Me.BackgroundImage = StateImages.MainImage
    End If
End Sub

Protected Overrides Sub OnMouseDown(ByVal mevent As
System.Windows.Forms.MouseEventArgs)
    If PanelVariables.ButtonsStyle = ButtonsStyle.Graphical Then
        'return if this button is the selected one (Don't change its state)
        If PanelVariables.ButtonsStyle = ButtonsStyle.Graphical Then
            If PanelVariables.LayoutVeiw = LayoutView.SingleView Then
                If PanelVariables.ButtonsStyle = ButtonsStyle.Graphical Then
                    If Me.InnerPanel.Visible = True Then
                        Exit Sub
                    End If
                End If
            End If
        End If
        Me.BackgroundImage = StateImages.DownImage
    End If
    MyBase.OnMouseDown(mevent)
End Sub

Protected Overrides Sub OnMouseUp(ByVal mevent As System.Windows.Forms.MouseEventArgs)
    If PanelVariables.ButtonsStyle = ButtonsStyle.Graphical Then
        'return if this button is the selected one (Don't change its state)
        If PanelVariables.ButtonsStyle = ButtonsStyle.Graphical Then
            If PanelVariables.LayoutVeiw = LayoutView.SingleView Then
                If PanelVariables.ButtonsStyle = ButtonsStyle.Graphical Then
                    If Me.InnerPanel.Visible = True Then
                        Exit Sub
                    End If
                End If
            End If
        End If
        Me.BackgroundImage = StateImages.OverImage
    End If
    MyBase.OnMouseUp(mevent)
End Sub

Protected Overrides Sub OnClick(ByVal e As System.EventArgs)
    MyBase.OnClick(e)
    ' if this button is the selected one set its
```

```
    ' background image to StateImages.SelecedImage image
    If PanelVariables.LayoutVeiw = LayoutView.SingleView Then
        If PanelVariables.ButtonsStyle = ButtonsStyle.Graphical Then
            If Me.InnerPanel.Visible = True Then
                Me.BackgroundImage = StateImages.SelecedImage
            End If
        End If
    End If
End Sub
```

You can explore the source code to see how it works. If you want to write and test the `Buttons` and `Panels` classes, you can do so and send me your changes to add to this article and get this control completed.

Points of interest

Do you know that when I decided to write this control, it took only half an hour to write the code and test it and when I decided to write it here on CodeProject, it took an year!!! Don't ask me why.

History

- 27 Sep 2011: Fixed two mistakes in the source code (article and source code files have been updated).

License

This article, along with any associated source code and files, is licensed under The Code Project Open License (CPOL)

Written By

Yasser Sobhy

Egypt

I am a software and web developer using C#, C/C++, VB.NET, PHP

And a lot of other programs and technologies
I also have a good knowledge of Photoshop

My favorite hobby is writing application to solve any problems I face and to provide web community and computer users with a new helping solutions.

Comments and Discussions

You must Sign In to use this message board.

Search Comments

First Prev Next

My vote of 5
Jamal Alqabandi 5-Oct-11 13:38

Re: My vote of 5
Yasser Sobhy 5-Oct-11 22:40

My vote of 5
kiran dangar 4-Oct-11 19:53

Re: My vote of 5
Yasser Sobhy 5-Oct-11 8:58

My vote of 4
Ed Nutting 30-Sep-11 10:17

My vote of 1
nobodyxxxxx 26-Sep-11 20:18

Re: My vote of 1
Yasser Sobhy 26-Sep-11 20:51

Re: My vote of 1
Ed Nutting 30-Sep-11 10:14

Refresh 1

General News Suggestion Question Bug Answer Joke Praise Rant Admin

Use Ctrl+Left/Right to switch messages, Ctrl+Up/Down to switch threads, Ctrl+Shift+Left/Right to switch pages.

Permalink
Advertise
Privacy
Cookies
Terms of Use

Layout: fixed | fluid

Article Copyright 2011 by Yasser Sobhy
Everything else Copyright © CodeProject, 1999-2022

Web01 2.8:2022-10-01:1